

The University of Azad Jammu and Kashmir, Muzaffarabad
Department of Software Engineering



Student Name	Abdul Hadi Kiani
Roll Number	2025-SE-11
Course Title	OOP
Course Code	CSC-521
Course Instructor	Engr. Zeeshan Rasheed
Program	BSc Software Engineering
Semester	Spring 2026
Session	2025-2029
Project No	01
Title	Airline Reservation and Management System
Assigned Date	14May-2026
Due Date	01June- 2026

Contents

1. Introduction.....	3
2. Class Hierarchy & Design.....	3
Flight.....	3
Derived Classes.....	3
Passenger.....	3
Ticket	3
Airline	3
3. UML Diagram.....	4
4. OOP Features Implemented.....	4
Abstraction.....	4
Inheritance.....	4
Polymorphism	4
Encapsulation.....	4
Operator Overloading	4
STL Usage	4
Exception Handling	4
File Persistence	4
5. Functional Requirements Covered.....	5
Flight Management	5
Passenger Management.....	5
Booking.....	5
Cancellation & Refund.....	5
Reports	5
Menu Interface	5
6. Design Decisions & Justification.....	5
Why abstract Flight?	5
Why STL containers?.....	5
Smart Pointers.....	5
8. Known Limitations & Future Work:.....	5
7. Sample Output Screenshot:	6
9. Conclusion	6

Airline Reservation & Flight Management System

Object-Oriented Programming - Problem Based Learning Report

1. Introduction

SkyLink Airways faced issues with manual spreadsheets: overbooking, lost passenger records, and incorrect refunds. To solve this, a console-based C++ application was developed using OOP principles. The system manages flights, passengers, bookings, and generates reports while ensuring data integrity and extensibility for future features.

2. Class Hierarchy & Design

The system follows a clear OOP hierarchy as per requirements:

Flight

Abstract base class with attributes: flightNumber, origin, destination, departureDateTime, totalSeats, availableSeats. Contains 2 pure virtual functions: `calculateBaseFare()` and `displayDetails()`.

Derived Classes

`DomesticFlight`, `InternationalFlight`, `CharterFlight`. Each implements its own pricing logic. `CharterFlight` includes a `contractHolder` attribute.

Passenger

Base class with derived classes: `EconomyPassenger`, `BusinessPassenger`, `FirstClassPassenger`. Each has different baggage allowance, loyalty multiplier, and refund rules.

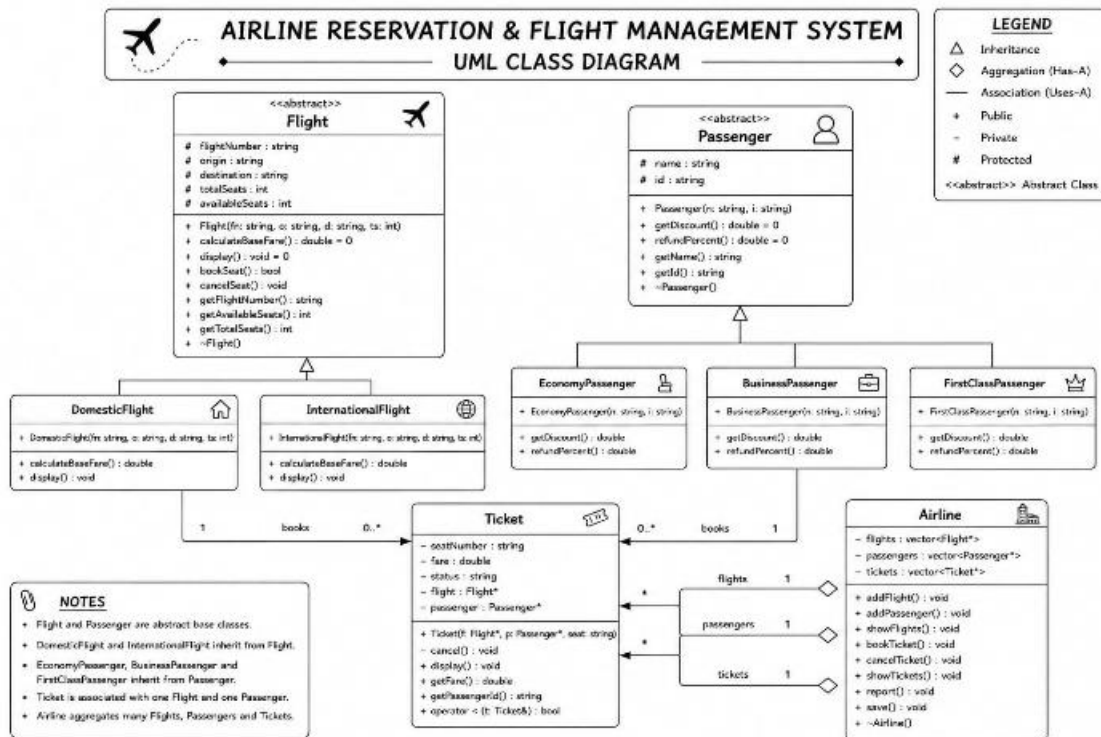
Ticket

Links a passenger to a flight. Stores seat number, fare paid, and booking status.

Airline

Aggregator class that holds collections of flights, passengers, and tickets. Exposes main operations like add, book, cancel, and report.

3. UML Diagram



4. OOP Features Implemented

As per the rubric, these features were used:

Abstraction

Flight is abstract with pure virtual functions to enforce implementation in derived classes.

Inheritance

Multi-level inheritance used. Example: 'CharterFlight → Flight', 'BusinessPassenger → Passenger'.

Polymorphism

Virtual functions allow runtime fare/refund calculation based on actual object type.

Encapsulation

All data members are private/protected with public getters/setters for controlled access.

Operator Overloading

'<<' operator overloaded for 'Flight' and 'Ticket' to enable direct printing via 'cout'

STL Usage

std::vector for storing flights/tickets, std::map for fast passenger lookup, std::sort for reports.

Exception Handling

Custom exceptions FlightFull.Exception and InvalidBookingException handle runtime errors gracefully.

File Persistence:

On exit, data saves to airline_data.txt. On startup, data loads back to maintain state.

5. Functional Requirements Covered

Flight Management

Add, remove, search by flight number/route/date, list all flights.

Passenger Management

Register passengers and view booking history.

Booking

Assigns available seat. Rejects if flight is full or passenger already has ticket.

Cancellation & Refund

Computes refund polymorphically based on passenger class and time before departure.

Reports

Generates "Today's departures", "Occupancy percentage per flight", and "Top 5 revenue flights".

Menu Interface

Clean console menu with input validation to prevent crashes.

6. Design Decisions & Justification

Why abstract Flight?

To enforce a common interface while letting each flight type define its own fare logic. Makes adding new flight types easy.

Why STL containers?

vector provides dynamic sizing for unknown number of flights. map gives $O(\log n)$ search for passengers.

Smart Pointers

Used `std::unique_ptr` in Airline class to prevent memory leaks and follow Rule of Five.

Separation of Concerns: Logic, data, and UI are separated. Airline handles logic, classes hold data, `main()` handles menu.

8. Known Limitations & Future Work:

1. File storage uses text format. Binary serialization would improve performance for large datasets.
2. No GUI - console based as per assignment. Can be upgraded to Qt or web interface later.
3. Single-user system. Multi-threading needed for concurrent bookings.

7. Sample Output Screenshot:

C:\Users\HP 348 G3\Desktop\Airline Registration System .exe

```

9. Save Data
0. Exit

Enter Choice: 1

1. Domestic Flight
2. International Flight
3. Charter Flight
Enter choice: 2
Flight Number: 21
Origin: Islamabad
Destination: Karachi
Total Seats: 30
Flight added successfully.

=====
= AIRLINE RESERVATION SYSTEM =
=====

1. Add Flight
2. Add Passenger
3. Show Flights
4. Show Passengers
5. Book Ticket
6. Show Tickets
7. Cancel Ticket
8. Occupancy Report
9. Save Data
0. Exit

Enter Choice:

```

```

project Classes ticket.h flight.h flight.cpp [*] main.cpp Airline.cpp Airline.h passenger.cpp passenger.h ticket.cpp
AirlineRegistration
├── Airline.cpp
├── Airline.h
├── flight.cpp
├── flight.h
├── main.cpp
├── passenger.cpp
├── passenger.h
├── ticket.cpp
└── ticket.h
1 #include <iostream>
2 #include "Airline.h"
3
4 using namespace std;
5
6 int main()
7 {
8     Airline airline;
9     int choice;
10
11     do
12     {
13         cout << "\n===== ";
14         cout << "\n= AIRLINE RESERVATION SYSTEM = ";
15         cout << "\n===== ";
16         cout << "\n1. Add Flight";
17         cout << "\n2. Add Passenger";
18         cout << "\n3. Show Flights";
19         cout << "\n4. Show Passengers";
20         cout << "\n5. Book Ticket";
21         cout << "\n6. Show Tickets";
22         cout << "\n7. Cancel Ticket";
23         cout << "\n8. Occupancy Report";
24         cout << "\n9. Save Data";
25         cout << "\n0. Exit";
26         cout << "\n\nEnter Choice: ";

```

9. Conclusion

This project strengthened understanding of core OOP concepts. Polymorphism and abstraction made the system extensible. The modular design ensures new features like new passenger types or payment methods can be added with minimal changes. The goal of applying engineering thinking was achieved.